# CubeGraph: A Multi-Resolution Graph Index for Approximate $k$-Nearest Neighbor Search with Geometric Filters

Mingyu Yang
HKUST (GZ)
China
myang250@connect.hkust-gz.edu.cn

Wentao Li
University of Leicester
United Kingdom
wl226@leicester.ac.uk

Wenxuan Xia
HKUST (GZ)
China
wxia248@connect.hkust-gz.edu.cn

Wei Wang
HKUST (GZ) & HKUST
China
weiwcs@ust.hk

## ABSTRACT

In real-world vector databases, objects are often associated with numerical attributes. This motivates filtered approximate $k$-nearest neighbor (A$k$NN) search to retrieve nearest neighbors among objects satisfying specified attribute constraints. Existing methods either adapt indexes designed for unfiltered A$k$NN search or incorporate attribute information into index construction. However, these methods typically assume regular filter regions, such as axis-aligned hyperrectangles, whereas real-world applications often involve arbitrary geometric regions, such as geographic boundaries for points of interest (POIs). Supporting such geometric filters introduces a fundamental trade-off between selectivity and navigability. Specifically, existing methods either retain excessive filter-mismatched objects to preserve navigation paths, reducing selectivity, or exclude non-matching objects that provide essential navigation paths, compromising search accuracy.

To address this challenge, we propose CubeGraph, a multi-resolution graph index for A$k$NN search with geometric filters. The key idea of CubeGraph is to partition the attribute space into multiple layers, each covering the entire space with hyperrectangular cells. Upper layers contain fewer but larger cells, while lower layers contain more but smaller cells, enabling query-adaptive layer selection to balance selectivity and navigability. We theoretically bound the selectivity of the selected cells for a given query and further enhance navigability by introducing edges between adjacent cells. Moreover, we develop two search strategies, precomputed and on-the-fly, to accommodate different selectivity–navigability trade-offs. Experiments on four real-world datasets containing up to 10 million vectors demonstrate that CubeGraph supports diverse geometric filters and achieves competitive or superior recall–throughput trade-offs compared with existing methods.

**PVLDB Reference Format:**
Mingyu Yang, Wentao Li, Wenxuan Xia, and Wei Wang. CubeGraph: A Multi-Resolution Graph Index for Approximate $k$-Nearest Neighbor Search with Geometric Filters. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

**PVLDB Artifact Availability:**
The source code, data, and/or other artifacts have been made available at https://github.com/mingyu-hkustgz/CubeGraph.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

## 1 INTRODUCTION

In many real-world AI applications, objects such as images and text are represented as high-dimensional vector embeddings that capture their semantic information [27, 29]. Given a dataset $\mathcal{D}$ of vector embeddings and a query vector $q$, $k$-nearest neighbor ($k$NN) search retrieves the $k$ vectors in $\mathcal{D}$ closest to $q$. However, exact $k$NN search is computationally expensive in high-dimensional spaces due to the curse of dimensionality [14]. To improve efficiency, **approximate $k$NN (A$k$NN) search** is widely adopted, trading some loss in accuracy for faster query processing [4]. A$k$NN search is widely used in applications such as retrieval-augmented generation for large language models and recommendation systems.

In real-world applications, objects are often associated with additional numerical attributes, such as locations and timestamps [2]. Accordingly, each object $o_i \in \mathcal{D}$ is represented by a vector embedding $x_i$ and numerical attributes $s_i$. Users can specify an attribute filter $\phi$, which defines a geometric region in the attribute space, to restrict retrieval to *filter-matching objects* whose attributes fall within $\phi$. This gives rise to **filtered A$k$NN search**, which retrieves approximate nearest neighbors among objects satisfying $\phi$. Such queries enable more flexible retrieval, such as finding images captured within a specified time range or geographic region.

**Geometric Filters.** Despite the growing importance of filtered A$k$NN search, existing studies primarily focus on *regular* geometric filters in the attribute space, such as one-dimensional (1D) intervals and low-dimensional hyperrectangles. This is largely because efficiently supporting even these simple filters remains challenging [2, 21, 43, 54]. However, real-world applications increasingly require more expressive filters involving multiple, potentially correlated attributes. For example, users may seek objects semantically similar to a query image within an irregular geographic region defined by natural or administrative boundaries, which cannot be precisely represented by conventional range filters.

Motivated by these requirements, we investigate **A$k$NN search with geometric filters**, extending conventional filtered A$k$NN search to support arbitrary geometric regions in numerical attribute spaces, including composite regions defined by inclusion and exclusion constraints. Given a query vector $\mathbf{q}$ and a geometric filter $\phi$, the goal is to retrieve the approximate $k$ nearest neighbors of $\mathbf{q}$ among objects in $\mathcal{D}$ whose attributes satisfy $\phi$. Such filters may specify intervals, hyperrectangles, circles, polygons, or arbitrary inclusion–exclusion regions. We illustrate the expressiveness of geometric filters through two representative applications.

*Urban Event Retrieval.* A city management platform stores geo-tagged texts, images, and videos, each represented by a vector

embedding with a timestamp and 2D location attributes. Given a query vector $\mathbf{q}$ representing an event such as *flooded streets*, an analyst seeks semantically similar objects captured within a specified time window and an irregular flood-affected region. This defines an irregular region in the 3D attribute space that cannot be precisely represented by conventional axis-aligned range filters. Approximating it with a bounding box may introduce numerous irrelevant objects, incurring additional filtering costs.

*Regional Multimedia Retrieval.* A multimedia database $\mathcal{D}$ stores geo-tagged images and videos, each represented by a vector embedding with a 1D timestamp and 2D location attributes. Given a query image $\mathbf{q}$, a user seeks visually similar objects captured within a specified time interval and a complex geographic polygon, while excluding restricted subregions such as airports or military zones. Such composite *inclusion–exclusion* filters enable expressive retrieval constraints that cannot be directly represented by conventional range filters.

**Existing Solutions.** Despite its greater expressiveness, A$k$NN search with geometric filters is challenging because it requires jointly handling high-dimensional vector similarity search and complex geometric constraints over multidimensional attributes. A natural approach is to adapt existing filtered A$k$NN methods designed for regular attribute filters, which can be broadly classified into two categories: *filtering-based* and *partition-based* methods.

*Filtering-based Methods.* These methods rely on conventional A$k$NN indexes that do not explicitly incorporate attribute information. Among them, graph-based indexes (e.g., HNSW [27]) are widely adopted for their state-of-the-art query performance [4, 9, 38]. Such indexes represent vectors as graph nodes connected by edges, enabling efficient similarity search through graph traversal. To support attribute filters, existing methods typically employ either PreFiltering or PostFiltering strategies. PreFiltering excludes filter-mismatching nodes during traversal, potentially disrupting graph connectivity and degrading search accuracy. In contrast, PostFiltering traverses the graph without enforcing attribute constraints and filters candidates afterward, incurring redundant distance computations when many candidates fail the filter. Methods such as ACORN [29] improve connectivity under attribute filtering by constructing denser graphs with additional edges and dynamically forming a subgraph of filter-matching nodes during search.

*Partition-based Methods.* These methods incorporate attribute information by partitioning the attribute space to organize vectors and enable pruning during filtered A$k$NN search [7, 8, 13, 16, 28, 29, 34, 40–43, 45, 48, 51, 53, 54]. For one-dimensional attributes (e.g., timestamps), methods such as WindowFilter [8], iRangeGraph [43], ESG [45], and WoW [40] employ segment-tree-like structures to organize vectors by attribute values. Queries are processed by searching tree nodes whose attribute ranges overlap the filter. For multidimensional attributes, Mesh [33] employs R-tree-based spatial partitioning and searches the local graph indexes associated with intersecting tree nodes. More recently, KHI [50] adopts KD-tree-based spatial partitioning and dynamically constructs a search graph containing only filter-matching vectors.

**Limitations.** For A$k$NN search with geometric filters, existing methods face a fundamental conflict between *selectivity* and *navigability*: excluding filter-mismatching objects improves selectivity but compromises navigability, while preserving navigability requires exploring more irrelevant objects. These methods fall into three groups with increasing selectivity.

(1) Filtering-based methods such as PostFiltering offer the lowest selectivity, as they traverse the original graph without excluding filter-mismatching objects. Although this preserves graph navigability, it incurs substantial overhead from visiting irrelevant objects.

(2) Partition-based methods, including WindowFilter [8] and ESG [45] for 1D attributes and Mesh [33] for multidimensional attributes, improve selectivity by pruning tree nodes that do not intersect the filter. However, searching separate local graphs associated with selected tree nodes limits cross-partition navigation, potentially degrading search accuracy.

(3) Methods such as PreFiltering achieve the highest selectivity by excluding all filter-mismatching objects during traversal, but may disrupt graph connectivity. To address this issue, iRangeGraph [43] for 1D attributes and KHI [50] for multidimensional attributes construct connected search graphs over filter-matching objects. Nevertheless, these graphs may still suffer from poor navigability and local optima, as they exclude filter-mismatching objects that could serve as intermediate nodes for reaching better neighbors. Consequently, existing methods struggle to achieve both high selectivity and effective graph navigation under complex geometric filters.

**Our Solution.** To address the conflict between selectivity and navigability, we propose CubeGraph, a multi-resolution graph index for A$k$NN search with geometric filters. The key idea is to partition the attribute space into a hierarchy of $L$ layers, each comprising hyperrectangular cells. Upper layers contain fewer, coarser cells, while lower layers progressively subdivide them into more, finer cells. Each cell organizes objects whose attributes fall within its boundaries using a local graph index. Although spatial partitioning itself is not new, we show that this multi-resolution design provides an effective mechanism for balancing selectivity and navigability.

(1) *Query-adaptive layer selection.* Coarser layers offer better navigability through larger local graphs but lower selectivity, as their cells may contain more filter-mismatching objects. Conversely, finer layers improve selectivity by reducing irrelevant objects within cells, but their smaller, fragmented graphs limit navigability. To balance these factors, CubeGraph selects an appropriate layer based on the characteristic length of the geometric filter, defined as the maximum side length of its bounding hyperrectangle. We theoretically analyze the selectivity of the selected cells, providing a foundation for this layer selection strategy.

(2) *Cross-cell graph navigation.* To improve navigability, CubeGraph augments each layer with cross-cell edges connecting objects in adjacent cells. These edges enable traversal across local graphs, facilitating exploration beyond individual cells and helping the search escape local optima.

(3) *Adaptive search strategies.* Given a selected layer, CubeGraph offers two search strategies to further balance selectivity and navigability. The *precomputed strategy* identifies all cells intersecting the filter and searches their interconnected local graphs, improving navigability but potentially requiring additional cells containing

only filter-mismatching objects to maintain connectivity. In contrast, the *on-the-fly strategy* dynamically explores adjacent cells only if they intersect the filter, improving selectivity by excluding irrelevant cells from graph traversal.

**Contributions.** Our main contributions are summarized as follows.

*Problem Formulation and Limitation Analysis (§ 2–3).* We formally define A$k$NN search with geometric filters and systematically analyze existing filtered A$k$NN methods, revealing their fundamental trade-off between selectivity and navigability.

*Multi-Resolution Graph Index (§ 4).* We propose CubeGraph, a multi-resolution graph index that balances selectivity and navigability through hierarchical spatial partitioning. Each layer maintains local graphs with cross-cell connections, enabling query-adaptive layer selection. We further develop precomputed and on-the-fly search strategies to accommodate different geometric filters.

*Index Construction and Maintenance (§ 5).* We develop efficient index construction algorithms that reuse graph structures across layers to reduce construction overhead. We also design incremental maintenance algorithms supporting object insertions and deletions without rebuilding the index from scratch.

*Experimental Evaluation (§ 6).* We conduct extensive experiments on four real-world datasets containing up to 10 million vectors. The results demonstrate that CubeGraph supports diverse geometric filters and achieves competitive or superior search performance compared with existing methods.

Due to space limitations, proofs and additional experiments are provided in our technical report [47].

## 2 PRELIMINARIES

We formally define the approximate $k$-nearest neighbor (A$k$NN) search problem with geometric filters in § 2.1, followed by the necessary technical background in § 2.2.

### 2.1 Problem Statement

Let $\mathcal{D} = \{o_i\}_{i=1}^{N}$ be a dataset of $N$ objects, where each object $o_i = (\mathbf{x}_i, \mathbf{s}_i)$ consists of a $d$-dimensional embedding $\mathbf{x}_i \in \mathbb{R}^d$ capturing semantic information and an $m$-dimensional attribute vector $\mathbf{s}_i \in \mathbb{R}^m$ describing numerical attributes. For example, an image may be represented by a CLIP embedding and a spatiotemporal attribute vector. We focus on numerical attributes that admit a geometric interpretation, leaving categorical attributes for future work. Users can specify attribute filters to restrict search results to objects satisfying their constraints.

*Definition 2.1 (Filter).* A *filter* is a Boolean function $\phi : \mathcal{S} \to \{0, 1\}$ over an $m$-dimensional attribute space $\mathcal{S} \subseteq \mathbb{R}^m$. An object $o_i = (\mathbf{x}_i, \mathbf{s}_i)$ satisfies $\phi$ if and only if $\phi(\mathbf{s}_i) = 1$.

Without loss of generality, we assume that filters are defined over all $m$ attribute dimensions, as filters involving only a subset can leave the remaining dimensions unconstrained. We define the set of objects satisfying a filter as follows.

*Definition 2.2 (Filtered Set).* Given a dataset $\mathcal{D}$ and a filter $\phi$, the *filtered set* is $\mathcal{D}_\phi = \{o_i = (\mathbf{x}_i, \mathbf{s}_i) \in \mathcal{D} \mid \phi(\mathbf{s}_i) = 1\}$, where each object in $\mathcal{D}_\phi$ is called a *filter-matching object*.

**Table 1: Summary of Notations**

| Notation | Description |
|---|---|
| $\mathcal{D}$ | Dataset containing $N$ objects |
| $o_i$ | Object $o_i = (\mathbf{x}_i, \mathbf{s}_i)$ |
| $N$ | Number of objects |
| $d$ | Embedding dimensionality |
| $m$ | Attribute dimensionality |
| $\mathbf{x}_i$ | Embedding vector of object $o_i$ |
| $\mathbf{s}_i$ | Attribute vector of object $o_i$ |
| $\mathcal{S}$ | Attribute space |
| $\mathbf{q}$ | Query vector |
| $\phi$ | Geometric filter over $\mathcal{S}$ |
| $B(\phi)$ | Tight axis-aligned bounding box enclosing $\mathcal{D}_\phi$ |
| $len(\phi)$ | Maximum side length of $B(\phi)$ |
| $k$ | Number of nearest neighbors |
| $\mathcal{D}_\phi$ | Objects satisfying $\phi$ |
| $\mathcal{R}$ | Approximate top-$k$ result set |

Geometrically, a filter $\phi$ defines a region in the attribute space, such as a hyperrectangle, polygon, temporal window, or Boolean combination thereof. Let $B(\phi)$ denote the minimum axis-aligned bounding box enclosing this region. We define the *characteristic length* $len(\phi)$ as the maximum side length of $B(\phi)$. The filtered set $\mathcal{D}_\phi$ thus consists of all objects whose attribute vectors lie within the region defined by $\phi$.

*Example 2.3.* Consider a dataset $\mathcal{D}$ of four objects, each associated with an embedding $\mathbf{x}_i$ and an attribute vector $\mathbf{s}_i$. Suppose a filter $\phi$ defines an irregular region with bounding box $B(\phi)$ of side lengths $b_1$ and $b_2$, where $b_1 > b_2$. Then, $len(\phi) = b_1$. If only $o_1$ and $o_3$ satisfy $\phi$, the filtered set is $\mathcal{D}_\phi = \{o_1, o_3\}$.

**Problem Definition.** Given a dataset $\mathcal{D}$ and a query $Q = (\mathbf{q}, \phi)$, where $\mathbf{q} \in \mathbb{R}^d$ is a query vector and $\phi$ is an attribute filter, **filtered A$k$NN search** retrieves a set $\mathcal{R} \subseteq \mathcal{D}_\phi$ of $k$ approximate nearest neighbors to $\mathbf{q}$, restricting the search to filter-matching objects $\mathcal{D}_\phi$. Existing studies primarily consider filters defined by axis-aligned hyperrectangles or hyperspheres. However, real-world applications often require more flexible geometric constraints, where regions of interest may have arbitrary shapes that cannot be directly captured by such filters. To accommodate these constraints, we consider *geometric filters*, which define arbitrarily shaped regions in the numerical attribute space. We thus study *AkNN search with geometric filters*, a special case of filtered A$k$NN search that supports arbitrary geometric regions. Unless otherwise specified, we measure vector similarity using Euclidean distance, although other measures, such as inner product, can be supported by the underlying A$k$NN index. For simplicity, we omit $k$ from the query notation. Table 1 summarizes the notation used throughout the paper.

*Example 2.4.* Consider a query $Q = (\mathbf{q}, \phi)$ with $k = 1$ and $\mathcal{D}_\phi = \{o_1, o_3\}$. Suppose $o_2$ is the nearest neighbor to $\mathbf{q}$ in $\mathcal{D}$ but does not satisfy $\phi$. If $o_3$ is closer to $\mathbf{q}$ than $o_1$, then $o_3$ is the nearest filter-matching neighbor and thus the desired result.

### 2.2 Technical Foundations

We review two technical foundations relevant to our approach: plain A$k$NN search without attribute filters and index merging over partitioned datasets.

**Plain A$k$NN Search.** Plain A$k$NN search retrieves approximate nearest neighbors in a high-dimensional vector space without attribute constraints. Existing methods fall into three main categories: hashing-, quantization-, and graph-based approaches. Hashing methods, such as locality-sensitive hashing (LSH) [12], map nearby vectors to the same buckets with high probability, enabling efficient candidate retrieval. Quantization methods [3, 5, 10, 11, 19, 46] compress vectors into compact representations to reduce storage costs and accelerate distance computation. Graph-based methods construct proximity graphs over $\mathcal{D}$ and perform graph traversal to identify approximate nearest neighbors. Representative indexes include HNSW [27], NSG [9], HVS [24], tMRNG [30], DiskANN [15], and others [20, 36, 37, 49]. Given their strong search performance, we adopt graph-based indexes for vector subsets in our approach.

**Index Merging.** When a dataset is partitioned, separate indexes can be constructed for individual partitions. Index merging combines these independently constructed indexes into a unified searchable structure, avoiding the cost of rebuilding a monolithic index. This technique is particularly useful in disk-based, distributed, and heterogeneous vector search systems [15, 53]. Existing methods typically merge indexes through overlapping objects or cross-partition connections. For example, DiskANN [15] employs overlapping objects across partitions, while HNSW merging [17] connects independently constructed graphs through shared objects. FGIM [1] accelerates graph merging using an improved NN-Descent algorithm, whereas RNSM [18] establishes cross-partition connections by identifying nearest neighbors across partitions. These techniques motivate our approach to enhancing graph navigability across partitions in a partitioned vector dataset.

## 3 PROBLEM ANALYSIS

A$k$NN search with geometric filters can potentially leverage existing methods for filtered A$k$NN search. Note that existing methods often focus on attribute filters with axis-aligned ranges or hyperrectangles. We first review these methods in § 3.1 and then discuss the limitations of extending them to arbitrary geometric filters in § 3.2, motivating our proposed approach.

### 3.1 Existing Solutions

We categorize existing methods for filtered A$k$NN search into *filtering-based* and *partition-based* approaches. The former apply attribute filters to general-purpose indexes, whereas the latter explicitly incorporate attributes into index construction.

**Filtering-based Methods.** Filtering-based methods often construct general-purpose graph indexes (e.g., HNSW) without considering attributes and apply filters during or after search. Two representative strategies are PreFiltering and PostFiltering [13]. PreFiltering excludes filter-mismatching objects during graph traversal, reducing unnecessary exploration. However, pruning these objects may fragment the search graph under selective filters, degrading search accuracy. In contrast, PostFiltering traverses the original graph without attribute restrictions, preserving navigability but potentially visiting numerous filter-mismatching objects and incurring substantial search overhead.

Several methods extend these strategies to better handle attribute constraints. For categorical filters, Filtered-DiskANN [13] constructs label-aware subgraphs to support category-specific searches. For low-dimensional numerical attributes, NHQ [35] incorporates both attribute proximity and vector similarity into graph construction. More recently, ACORN [29] augments HNSW with additional edges and dynamically expands neighborhoods to support filtered traversal. At query time, ACORN constructs a filter-aware search graph by selecting filter-matching neighbors from the augmented index, reducing exploration of irrelevant objects while improving connectivity. However, its expanded neighborhoods may still provide insufficient connectivity, as they cannot cover all filter-matching objects. Moreover, excluding filter-mismatching objects eliminates potentially useful intermediate paths to the nearest neighbors, which may further compromise search accuracy.

**Partition-based Methods.** Partition-based methods organize the attribute space into subregions and construct local graph indexes (e.g., HNSW) over the objects associated with each subregion. For one-dimensional range filters, methods such as SeRF [54], WindowFilter [8], iRangeGraph [43], ESG [45], and WoW [40] typically employ segment-tree-like structures, with each node maintaining a graph index over a subset of objects. A query range can be decomposed into $O(\log N)$ canonical segments, each corresponding to a tree node. Methods such as SeRF and WindowFilter independently search the corresponding local indexes and merge their results, potentially exploring numerous filter-mismatching objects. To reduce such exploration, iRangeGraph dynamically constructs a filter-aware search graph by restricting neighbor selection to filter-matching objects. However, these methods primarily target one-dimensional attributes and may incur substantial index storage overhead due to $O(\log N)$ object replication across tree nodes.

For multi-dimensional attributes, spatial hierarchies such as R-trees [6] and KD-trees [32] partition the attribute space into subregions. Mesh [33] employs an R-tree whose nodes maintain local graph indexes. At query time, it identifies nodes overlapping the filter region and independently searches their local indexes, potentially visiting many filter-mismatching objects. To mitigate this overhead, KHI [50] extends iRangeGraph to multi-dimensional attributes using a KD-tree with graph indexes maintained at tree nodes. Given a query, KHI identifies filter-matching objects as entry points and dynamically constructs a search graph by traversing the tree from the root to relevant nodes, restricting neighbor selection to filter-matching objects.

### 3.2 Limitations

When extended to geometric filters, existing methods face an inherent trade-off between *selectivity* and *navigability*. Selectivity measures how effectively a method avoids visiting filter-mismatching objects, while navigability reflects how effectively its graph indexes guide the search toward nearest neighbors. Existing methods exhibit three distinct trade-offs.

**(1) Low Selectivity, High Navigability.** Methods such as PostFiltering traverse the original graph without attribute restrictions, potentially visiting numerous filter-mismatching objects. However, they preserve the graph's original connectivity, allowing filter-mismatching objects to serve as intermediate nodes along

search paths toward the results. This maintains strong navigability at the cost of low selectivity.

**(2) Moderate Selectivity and Navigability.** Partition-based methods, including WindowFilter [8] and ESG [45] for one-dimensional attributes and Mesh [33] for multi-dimensional attributes, organize the attribute space using hierarchical structures such as segment trees and R-trees. Each tree node maintains a local graph index over objects in its corresponding subregion. At query time, these methods identify nodes whose subregions overlap the filter and independently search their local indexes before merging the results. By pruning irrelevant subregions, they improve selectivity over PostFiltering, although overlapping nodes may still contain filter-mismatching objects. Meanwhile, retaining these objects within local indexes preserves auxiliary search paths, but independent searches across partitions may limit global navigability.

**(3) High Selectivity, Low Navigability.** Methods such as PreFiltering achieve high selectivity by excluding all filter-mismatching objects during traversal. However, removing these objects may fragment the search graph and eliminate intermediate paths to the results, substantially degrading navigability. To mitigate this issue, iRangeGraph [43] and KHI [50] organize the attribute space using segment-tree-like and KD-tree structures, respectively, and dynamically construct filter-aware search graphs by selecting only filter-matching neighbors. Although this improves connectivity among eligible objects compared with PreFiltering, these methods still exclude filter-mismatching objects that could provide useful intermediate paths. Thus, their search graphs may exhibit limited navigability, particularly under complex filters.

**Summary.** Existing methods face an inherent trade-off between selectivity and navigability. Although their impact on search efficiency and accuracy also depends on index structures and query characteristics, low selectivity generally incurs unnecessary exploration of filter-mismatching objects, while poor navigability may degrade recall by limiting access to relevant neighbors. Moreover, existing partition-based methods often employ hierarchical structures that replicate objects across multiple levels, potentially incurring substantial storage overhead (e.g., $O(\log N)$ copies per object in iRangeGraph). These limitations motivate a new approach that (1) adaptively balances selectivity and navigability according to query characteristics and (2) provides user-configurable index sizes to control storage overhead. We present our solution in § 4.

# 4 OUR PROPOSED INDEX AND QUERY PROCESSING

This section introduces the proposed CubeGraph index (§ 4.1), its query processing algorithm (§ 4.2), and complexity analysis (§ 4.3).

## 4.1 The CubeGraph Index

To address the selectivity–navigability trade-off discussed in § 3.2, we propose CubeGraph, a multi-resolution graph index for A$k$NN search with geometric filters. The key idea is to enable query-adaptive graph selection that balances the exploration of filter-mismatching objects against graph navigability.

**Index Structure.** Given a dataset $\mathcal{D} = \{o_i = (\mathbf{x}_i, \mathbf{s}_i)\}_{i=1}^{N}$, we normalize each attribute dimension to $[0, 1]$, yielding an attribute space $\mathcal{B} = [0, 1]^m$. CubeGraph organizes $\mathcal{B}$ into a hierarchy of $L$ layers, where $L$ is a user-defined parameter controlling index size. Each layer covers the entire attribute space, with coarser partitions at upper layers and progressively finer partitions at lower layers. This multi-resolution design enables query-adaptive layer selection to balance selectivity and navigability, rather than relying on searches across multiple overlapping tree nodes.

*Intra-Cell Index.* At each layer $\ell$ ($0 \le \ell < L$), $\mathcal{B}$ is uniformly partitioned into $g_\ell^m$ axis-aligned hypercubic cells/cubes, where $g_\ell = 2^{\ell+1}$ and each cell has side length $w_\ell = 1/g_\ell$. Thus, increasing $\ell$ yields more, smaller cells. Each cell is identified by its grid coordinate $\mathbf{a} = (a_1, \ldots, a_m) \in \{0, \ldots, g_\ell - 1\}^m$ and covers $\prod_{r=1}^{m} [a_r w_\ell, (a_r + 1)w_\ell)$, with the upper boundary 1 included in the last interval of each dimension. Each object belongs to exactly one cell per layer, determined by its attribute vector. For each non-empty cell, CubeGraph constructs a bounded-degree local proximity graph (e.g., HNSW) over its objects, where vertices represent objects and edges support efficient intra-cell navigation.

*Cross-Cell Connections.* To improve navigability across cells, CubeGraph introduces edges between face-adjacent cells at each layer. Two cells $c_{\ell,\mathbf{a}}$ and $c_{\ell,\mathbf{b}}$ are *face-adjacent* if $\|\mathbf{a}-\mathbf{b}\|_1 = 1$, meaning they differ by one grid step along exactly one attribute dimension and share an $(m-1)$-dimensional face. Each object additionally maintains edges to its $M_{\text{cross}}$ nearest neighbors in each non-empty face-adjacent cell. These cross-cell edges facilitate navigation between neighboring cells, although global connectivity also depends on the connectivity of local graphs and the adjacency structure of occupied cells. We formally define the CubeGraph index as follows.

*Definition 4.1 (*CubeGraph *Index).* Given a dataset $\mathcal{D}$ and a user-defined number of layers $L$, the CubeGraph index is defined as

$$\text{CubeGraph} = \{(C_\ell, \mathcal{G}_\ell^{\text{intra}}, \mathcal{G}_\ell^{\text{cross}})\}_{\ell=0}^{L-1},$$

where $C_\ell$ denotes the set of non-empty cells at layer $\ell$, $\mathcal{G}_\ell^{\text{intra}}$ contains their local proximity graphs, and $\mathcal{G}_\ell^{\text{cross}}$ contains cross-cell edges connecting objects in face-adjacent cells.

*Example 4.2.* The left panel of Fig. 1 shows the CubeGraph index over 2D (top) and 3D (bottom) attribute spaces. For the 2D case, layer 0 partitions the attribute space into a $2 \times 2$ grid, while layer 1 further refines it into a $4 \times 4$ grid, forming a multi-resolution hierarchy. At each layer, every non-empty cell maintains a local graph index over its objects, with cross-cell edges connecting objects in adjacent cells. The 3D case follows the same construction principle.

**Memory Layout.** We describe the physical memory layout of the logical components in Definition 4.1. At each layer $\ell$, the cell directory $C_\ell$ is implemented as a sparse array that stores only non-empty cells, mapping each cell coordinate to a contiguous range of object records. Since each object belongs to exactly one cell per layer, its intra-cell graph can be stored contiguously. As illustrated in Fig. 2, each object record contains its ID, attribute metadata, intra-cell neighbor list, and cross-cell neighbor lists. This layout supports the query processing in § 4.2: graph traversal first explores intra-cell neighbors in $\mathcal{G}_\ell^{\text{intra}}$ and accesses the corresponding cross-cell neighbor list in $\mathcal{G}_\ell^{\text{cross}}$ when traversing to an adjacent cell. Cross-cell neighbors are therefore grouped by destination cell rather than

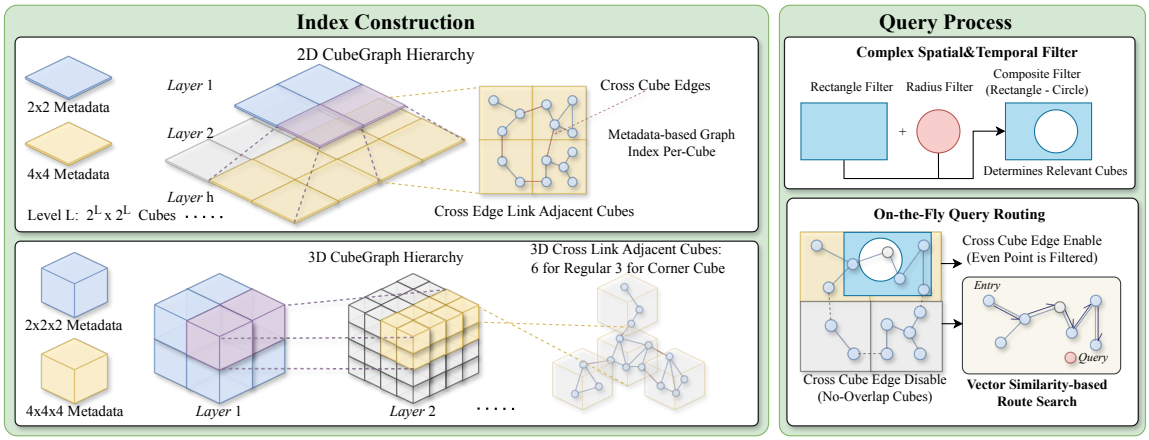



**Figure 1: Overview of** CubeGraph**. Left: Multi-resolution graph indexes over 2D and 3D attribute spaces, where each layer partitions the space into cells with local graph indexes and cross-cell connections. Right: Query processing for a rectangle-minus-circle filter, illustrating layer selection and graph traversal using on-the-fly search (here, layers are indexed from 1 for clarity, versus 0 in the main text).**

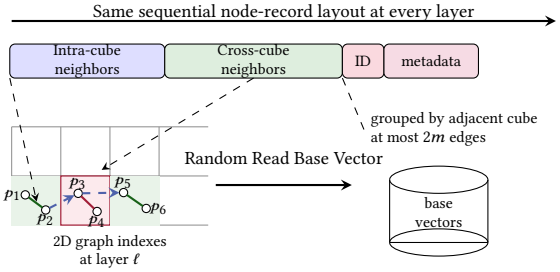



**Figure 2: Physical memory layout of the** CubeGraph **index.**

stored in a single list. The same layout applies across all layers, differing only in cell granularity.

*Example 4.3.* Consider object $p_2$ in Fig. 2. Its intra-cell neighbor list contains $p_1$, which belongs to the same cell, while its cross-cell neighbor list contains $p_5$ in the right face-adjacent cell. The record also stores the object ID and attribute vector.

**Index Size.** We analyze the space complexity of CubeGraph under the above memory layout.

Theorem 4.4. *Assume that* CubeGraph *materializes only non-empty cells and that the maximum intra-cell degree $M$ and the maximum cross-cell degree $M_{\text{cross}}$ per adjacent cell are constants. Then, the space cost of* CubeGraph *is $O(LNm)$, where $L$ is the number of layers, $N$ is the number of objects, and $m$ is the attribute dimensionality.*

The bound assumes constant degree parameters, as commonly adopted in graph-based A$k$NN indexes. Since each object is represented once per layer, the index size scales linearly with the user-configurable number of layers $L$. By default, we use at most six layers and stop further refinement when the average number of objects per cell falls below 50. The proof of Theorem 4.4 is provided in our technical report [47].

## 4.2 Query Processing

Given a query $Q = (\mathbf{q}, \phi)$, where $\mathbf{q}$ is the query vector and $\phi$ is a geometric filter, we aim to efficiently process $Q$ using our CubeGraph index. Recall that CubeGraph organizes the objects in $\mathcal{D}$ into a hierarchy of $L$ layers, each comprising local graphs interconnected by cross-cube edges (Definition 4.1). To process $Q$, we first select an appropriate layer and then perform graph search on the selected layer to retrieve the results. Accordingly, query processing consists of two phases: (1) layer selection and (2) graph search. Algorithm 1 outlines the overall procedure, with both phases detailed below.

**Algorithm 1:** CubeGraph Query Processing

**Input:** CubeGraph index, query vector $\mathbf{q}$, geometric filter $\phi$, result size $k$, search budget $ef$
**Output:** Top-$k$ objects satisfying $\phi$

1 $r \leftarrow \text{len}(\phi)$;
2 Select layer $\ell^*$ such that $r/2 < w_{\ell^*} \le r$; otherwise, select the nearest boundary layer;
3 **if** *$\phi$ supports efficient cell–filter intersection tests* **then**
4 $\quad$ **return** PredeterminedSearch$(\mathbf{q}, \phi, k, \ell^*, ef)$;
5 **else**
6 $\quad c_0 \leftarrow$ cell at layer $\ell^*$ containing a representative point of $\phi$;
7 $\quad$ **return** OnTheFlySearch$(\mathbf{q}, \phi, k, \ell^*, c_0, ef)$;

**Phase 1: Layer Selection.** Given a query $(\mathbf{q}, \phi)$, CubeGraph first selects an appropriate layer from its $L$-level hierarchy. Upper layers (starting from $\ell = 0$) contain fewer but larger cells, providing better navigability at the cost of lower selectivity due to more filter-mismatching objects. Conversely, lower layers (ending at $\ell = L - 1$) contain more but smaller cells, improving selectivity while potentially limiting navigability. To balance these competing factors, we propose a query-adaptive layer selection strategy based on the *characteristic length* $r = \text{len}(\phi)$, defined as the maximum side length of the bounding box enclosing $\phi$. The key idea is to select a layer whose cell width is comparable to $r$, allowing a cell to potentially cover the entire filter while limiting the inclusion of filter-mismatching objects and the number of intersecting cells.

Specifically, at layer $\ell$, the cell width is $w_\ell = 1/g_\ell$, where $g_\ell = 2^{\ell+1}$. As shown in Algorithm 1, we first compute $r = \text{len}(\phi)$ in the normalized attribute space (Line 1). We then select the layer $\ell^*$ whose cell width satisfies $r/2 < w_{\ell^*} \le r$ (Line 2). If no such layer exists, we select the boundary layer whose cell width is closest to

**Algorithm 2:** Predetermined Search

**Input:** Query vector $\mathbf{q}$, filter $\phi$, result size $k$, layer $\ell$, beam width $ef$
**Output:** Top-$k$ objects satisfying $\phi$

1 $R \leftarrow \emptyset$;
2 $C \leftarrow$ cells intersecting $\phi$ at layer $\ell$;
3 Initialize bitmap $B$ with $B[c] = 1$ for each $c \in C$;
4 Initialize priority queue $Q$ with entry points of cells in $C$;
5 **while** $Q \neq \emptyset$ ***and*** $(|R| < k$ ***or*** $d(\mathbf{q}, Q.\text{top}) < d(\mathbf{q}, R[k]))$ **do**
6  $p \leftarrow Q.\text{pop}()$;
7  **for each** *neighbor $u$ of $p$ via an intra- or cross-cell edge* **do**
8  **if** $(p, u)$ is a cross-cell edge **and** $B[u.\text{cell}] = 0$ **then continue**;
9  **if** $\phi(\mathbf{s}_u) = 1$ **then** Insert $u$ into $R$;
10  **if** $u$ is unvisited **then** Insert $u$ into $Q$;
11  Retain the closest $ef$ candidates in $Q$;
12 **return** Top-$k$ objects in $R$;

**Algorithm 3:** On-the-Fly Search

**Input:** Query vector $\mathbf{q}$, filter $\phi$, result size $k$, layer $\ell$, entry cell $c_0$, beam width $ef$
**Output:** Top-$k$ objects satisfying $\phi$

1 $R \leftarrow \emptyset$;
2 Initialize bitmap $B$ with $B[c_0] = 1$ and $B[c] = 0$ otherwise;
3 Initialize priority queue $Q$ with the entry point of $c_0$;
4 **while** $Q \neq \emptyset$ ***and*** $(|R| < k$ ***or*** $d(\mathbf{q}, Q.\text{top}) < d(\mathbf{q}, R[k]))$ **do**
5  $p \leftarrow Q.\text{pop}()$;
6  **for each** *neighbor $u$ of $p$ via an intra- or cross-cell edge* **do**
7  **if** $B[u.\text{cell}] = 1$ ***or*** $\phi(\mathbf{s}_u) = 1$ **then**
8  **if** $B[u.\text{cell}] = 0$ **then** $B[u.\text{cell}] \leftarrow 1$;
9  **if** $\phi(\mathbf{s}_u) = 1$ **then** Insert $u$ into $R$;
10  **if** $u$ is unvisited **then** Insert $u$ into $Q$;
11  Retain the closest $ef$ candidates in $Q$;
12 **return** Top-$k$ objects in $R$;

this range. Since $w_\ell$ decreases monotonically with $\ell$, the appropriate layer can be identified efficiently via binary search. We theoretically justify this selection criterion in § 4.3.

**Phase 2: Graph Search.** After selecting layer $\ell^*$, CubeGraph performs beam search over the graph formed by intra-cell and cross-cell edges at that layer. A key challenge is identifying the cells to explore during search. This can be achieved either by identifying all cells intersecting the query filter before traversal or by discovering them incrementally during traversal. The choice depends on whether $\phi$ supports efficient cell–region intersection tests.

Accordingly, CubeGraph provides two search modes: *predetermined search* and *on-the-fly search*. When $\phi$ supports efficient cell–region intersection tests, predetermined search (Lines 3–4 of Algorithm 1) identifies all intersecting cells before graph traversal, making it suitable for simple geometric filters such as axis-aligned boxes. Otherwise, on-the-fly search (Lines 5–7) selects an entry cell $c_0$ based on a representative point of $\phi$ and incrementally discovers relevant cells during traversal.

*Predetermined Search.* Algorithm 2 applies when cells intersecting $\phi$ can be identified efficiently. It first computes the cell cover $C$ of $\phi$ and records the intersecting cells in a bitmap $B$ (Lines 1–3). The search is then initialized using the entry points stored in the corresponding local graph indexes (Line 4). During traversal, each candidate object is expanded through both intra-cell and cross-cell edges (Lines 5–7). A cross-cell edge is traversed only if its target cell is marked in $B$ (Line 8), restricting the search to the precomputed cell cover. As intersecting cells may contain filter-mismatching objects, each visited object is checked against $\phi$ before being added to result set $R$ (Line 9). Unvisited candidates are inserted into the beam queue, bounded by the beam width $ef$ (Lines 10–11). Finally, the algorithm returns the top-$k$ filter-matching objects in $R$ (Line 12).

*On-the-Fly Search.* Algorithm 3 is used when enumerating all intersecting cubes is expensive. Instead, cube intersection tests are performed incrementally as new cubes are reached during graph traversal. It starts from an entry cube $c_0$ and maintains a dynamic bitmap $B$ of active cubes (Lines 1–3). During traversal, a neighbor $u$ is explored if it belongs to an active cube or satisfies $\phi$ (Lines 4–6). In the latter case, CubeGraph activates the cube containing $u$ by updating $B$ (Lines 7–8), enabling subsequent traversal within that cube and across its adjacent cubes. As in predetermined search, only objects satisfying $\phi$ are inserted into $R$, while unvisited searchable neighbors are added to the beam queue, whose size is bounded by $ef$ (Lines 9–11). Finally, top-$k$ valid objects in $R$ is returned.

*Example 4.5.* The right-hand panels of Fig. 1 illustrate query processing for a rectangular filter with a circular exclusion region. First, CubeGraph selects the layer whose cell width best matches the filter's characteristic length. At this layer, predetermined search marks all intersecting cells in bitmap $B$ before traversal and restricts the search to these cells. In contrast, on-the-fly search starts from an entry cell and incrementally activates new cells upon encountering filter-matching objects.

**Remark.** The two search modes differ in how they identify searchable cells, reflecting a trade-off between computational overhead and search accuracy. Predetermined search identifies all cells intersecting $\phi$ before traversal, potentially improving recall by ensuring their inclusion in the search, but incurs higher overhead when cell–filter intersection tests are expensive. In contrast, on-the-fly search avoids precomputing the cell cover by incrementally activating cells upon encountering filter-matching objects, but may miss relevant cells that are not reachable through the explored paths. To balance these trade-offs, we adopt predetermined search for axis-aligned box filters and on-the-fly search for other filters. Experiments in § 6 demonstrate the effectiveness of this hybrid strategy.

## 4.3 Analysis

We first bound the number of cells intersecting a query region under our layer selection strategy. We then analyze the fraction of filter-matching objects within these cells and derive a bound on query time. Together, these results provide theoretical justification for query processing in CubeGraph.

**Justification of Layer Selection.** Consider the normalized attribute space $\mathcal{S} = [0, 1]^m$. The following geometric bound on the number of intersecting cells is independent of the object distribution. At layer $\ell$, CubeGraph partitions each dimension into $g_\ell = 2^{\ell+1}$ intervals, yielding $g_\ell^m$ cells of width $w_\ell = 1/g_\ell$. For a filter $\phi$, let $r = \text{len}(\phi)$ denote the maximum side length of its tight bounding box. Whenever possible, CubeGraph selects the preferred layer $\ell^*$ satisfying $r/2 < w_{\ell^*} \leq r$. Since cell widths decrease by a factor of two between consecutive layers, such a layer exists whenever $r$ falls within the supported resolution range. Otherwise, CubeGraph selects the boundary layer whose cell width is closest to this range. The following analysis assumes that the preferred layer exists.

*Layer-Selection Guarantee.* Existing methods face an inherent trade-off between selectivity and navigability. CubeGraph mitigates this trade-off through a hierarchy of $L$ layers, selecting a layer $\ell^*$ whose cell width satisfies $r/2 < w_{\ell^*} \leq r$ for a query filter $\phi$, whenever such a layer exists. We first establish an upper bound on the number of cells intersecting $\phi$ to justify this selection criterion.

Proposition 4.6. *For a filter $\phi$ with characteristic length $r$, selecting a layer $\ell^*$ satisfying $r/2 < w_{\ell^*} \leq r$ guarantees that $\phi$ intersects at most $3^m$ cells.*

More generally, for a layer $\ell$ and a filter with bounding-box side lengths $a_i$, the number of intersected cells is bounded by $\prod_{i=1}^{m}(\lceil a_i/w_\ell \rceil + 1)$. Proposition 4.6 establishes an upper bound of $3^m$ intersected cells under the proposed layer-selection strategy. Yet, this bound does not directly characterize the number of filter-matching objects or the query cost, which we analyze next.

**Selectivity Analysis.** We further analyze the selectivity of the selected layer by quantifying the proportion of filter-matching objects among those contained in the intersected cells. To this end, we adopt the *elastic factor* [45] as a measure of selectivity. Let $C$ denote the set of cells intersecting the filter $\phi$ at the selected layer $\ell^*$, and let $\mathcal{G}^*$ contain all objects in their union $U = \bigcup_{C \in \mathcal{C}} C$. Since every filter-matching object belongs to an intersected cell, we have $\mathcal{D}_\phi \subseteq \mathcal{G}^*$, where $\mathcal{D}_\phi$ denotes the set of objects satisfying $\phi$.

*Definition 4.7 (Elastic Factor).* The *elastic factor* is defined as

$$e(\mathcal{D}_\phi, \mathcal{G}^*) = \frac{|\mathcal{D}_\phi|}{|\mathcal{G}^*|}.$$

The elastic factor measures the fraction of filter-matching objects among all objects in the intersected cells, rather than those actually visited during graph search. It thus characterizes the selectivity of the selected layer. For theoretical analysis, assume that $N$ objects are independently sampled from a fixed joint attribute distribution $\mu$, allowing correlations among attributes. For a fixed filter $\phi$ and partition with $\mu(U) > 0$, the law of large numbers yields

$$e(\mathcal{D}_\phi, \mathcal{G}^*) \xrightarrow[N\to\infty]{\text{a.s.}} \frac{\mu(\phi)}{\mu(U)},$$

where $\mu(A)$ denotes the probability mass of region $A$. Under a uniform attribute distribution, this limit reduces to the *volume elastic factor*

$$\bar{e}(\phi, \ell^*) = \frac{\mathrm{Vol}(\phi)}{\mathrm{Vol}(U)},$$

where Vol denotes geometric volume. We adopt the uniformity assumption for tractable theoretical analysis, leaving extensions to nonuniform distributions for future work. Under the uniformity assumption, we establish the following lower bound on the volume elastic factor for an $m$-dimensional ball filter of diameter $r$.

Lemma 4.8. *For an $m$-dimensional ball of diameter $r$ contained in $[0,1]^m$, under the layer-selection rule in Proposition 4.6, the volume elastic factor satisfies*

$$\bar{e} \geq \frac{\pi^{m/2}}{6^m \Gamma(m/2+1)}.$$

The bound decreases with the attribute dimensionality $m$. More generally, for an arbitrary geometric filter $\phi$ with characteristic length $r$, Proposition 4.6 yields $\bar{e} \geq \mathrm{Vol}(\phi)/(3^m r^m)$. Although these bounds characterize the selectivity of the intersected cells, they do not directly translate into query-time guarantees, as graph search need not visit all filter-matching objects to achieve a target recall. Nevertheless, they provide theoretical lower bounds on the proportion of relevant objects within the selected cells, supporting the effectiveness of our layer-selection strategy.

**Query-Time Complexity.** We analyze the query-time complexity of CubeGraph, which consists of three phases: (1) selecting the appropriate layer $\ell^*$; (2) identifying cells intersecting the query filter using either predetermined or on-the-fly search; and (3) performing beam search on the graph induced by the selected cells. Let $C$ denote the expected startup cost of beam search, abstracting away the implementation details of the underlying graph index. The overall complexity is bounded as follows.

Theorem 4.9. *Given a query $Q = (\mathbf{q}, \phi)$ with characteristic length $r$, suppose* CubeGraph *selects layer $\ell^*$. Assume that (1) cell discovery takes $T_{\text{cube}}$ time; (2) processing each distinct candidate, including distance computation, filter evaluation, bitmap operations, and queue maintenance, takes $O(1)$ time for fixed dimensions and search parameters; (3) the expected beam-search startup cost is $C$; and (4) until $k$ valid candidates are found, traversal can continue, with each newly examined candidate satisfying $\phi$ with conditional probability at least $c \in (0,1]$. Then, the expected time to retrieve $k$ valid candidates is*

$$O\left(\log L + T_{\text{cube}} + C + \frac{k}{c}\right).$$

*Interpretation.* Let $\mathcal{G}^*$ denote the graph induced by the selected cells and their cross-cube edges at layer $\ell^*$. Theorem 4.9 bounds the expected time to retrieve $k$ valid objects, rather than the cost of achieving a target A$k$NN recall. The startup cost $C$ depends on graph quality and query workload; it is often empirically observed to scale logarithmically with the number of objects in graph-based indexes [17, 18, 26, 29], although this is not a theoretical guarantee. For predetermined search, Proposition 4.6 yields $T_{\text{cube}} = O(3^m)$, assuming constant-time cube–filter intersection tests. For on-the-fly search, cube discovery is integrated into traversal through constant-time filter evaluation. Finally, the $k/c$ term captures filtering overhead: if each newly examined candidate has conditional validity probability at least a constant $c > 0$ until $k$ valid candidates are found, the expected post-startup search cost is $O(k)$.

# 5 INDEX CONSTRUCTION AND MAINTENANCE

We first describe the construction of the CubeGraph index in § 5.1, followed by its maintenance under dynamic updates in § 5.2.

## 5.1 Index Construction

Recall that CubeGraph consists of $L$ hierarchical layers, each comprising a sparse cell directory, intra-cell graphs, and cross-cell edges (Definition 4.1). For each layer $\ell$, index construction proceeds in two phases. Phase 1 partitions the attribute space into cells, assigns each object in $\mathcal{D}$ to a cell based on its attributes, and constructs the intra-cell graph $\mathcal{G}_\ell^{\text{intra}}$ within each non-empty cell. Phase 2 constructs the cross-cell graph $\mathcal{G}_\ell^{\text{cross}}$ by connecting objects in face-adjacent

**Algorithm 4:** Hierarchy Construction with Local Indexing

**Input:** Dataset $\mathcal{D}$, number of layers $L$, parameters $M$ and $ef_c$
**Output:** Local hierarchy $\mathrm{CI}^{\mathrm{Local}}$

1 Normalize attributes to $\mathcal{B} = [0,1]^m$;
2 **for** $\ell = 0$ **to** $L-1$ **do**
3  Set $g_\ell = 2^{\ell+1}$ and $w_\ell = 1/g_\ell$;
4  Partition $\mathcal{B}$ into $(g_\ell)^m$ cells;
5  **for each** $(\mathbf{x}_i, \mathbf{s}_i) \in \mathcal{D}$ ***in parallel*** **do**
6  Assign object $i$ to its cell based on $\mathbf{s}_i$;
7  **for each** *non-empty cell* $c$ ***in parallel*** **do**
8  Add $c$ to the sparse cell directory;
9  Build the local graph for $c$ using $M$ and $ef_c$;
10  Record the IDs of face-adjacent cells;

**Algorithm 5:** Cross-Cell Edge Construction

**Input:** Local hierarchy $\mathrm{CI}^{\mathrm{Local}}$, parameter $M_{\mathrm{cross}}$
**Output:** CubeGraph index

1 **for each** *non-empty cell* $c$ *at each layer* $\ell$ **do**
2  **for each** *object* $p \in c$ ***in parallel*** **do**
3  **for each** *non-empty face-adjacent cell* $c_{\mathrm{adj}}$ *of* $c$ **do**
4  Perform A$k$NN search in $c_{\mathrm{adj}}$ with query $p$;
5  Select up to $M_{\mathrm{cross}}$ nearest neighbors;
6  Add cross-cell edges from $p$ to the selected neighbors;

cells. Both phases are naturally parallelizable, as cell assignment, intra-cell graph construction, and cross-cell neighbor searches can be performed independently. We detail these two phases below.

**Phase 1: Local Graph Construction.** Algorithm 4 partitions the attribute space into cells, constructs sparse cell directories, and builds local graph indexes for each layer $\ell$. Given a dataset $\mathcal{D}$, we first normalize each attribute dimension to the unit domain $\mathcal{B} = [0,1]^m$ (Line 1), as described in § 4.1. For each layer $\ell \in [0, L-1]$, we set $g_\ell = 2^{\ell+1}$ and partition $\mathcal{B}$ into $(g_\ell)^m$ equal-sized cells with side length $w_\ell = 1/g_\ell$ (Lines 2–4). Each object is then assigned to exactly one cell based on its normalized attribute vector (Lines 5–6). Only non-empty cells are materialized and stored in the sparse cell directory. For each such cell, we construct a bounded-degree local graph using HNSW with parameters $M$ and $ef_c$, following the standard construction procedure [27] (Lines 7–8). Finally, we record the IDs of face-adjacent cells for Phase 2. Each $m$-dimensional cell has at most $2m$ face-adjacent neighbors, each differing by one grid step along a single dimension (Line 9–10).

**Phase 2: Cross-Cell Edge Construction.** After Phase 1, each layer contains disconnected local graphs. Algorithm 5 constructs the cross-cell graph $\mathcal{G}_\ell^{\mathrm{cross}}$ to enable traversal between neighboring cells and improve navigability. Specifically, for each layer $\ell$, object $p$ in cell $c$, and non-empty face-adjacent cell $c_{\mathrm{adj}}$, CubeGraph performs a standard HNSW beam search on the local graph of $c_{\mathrm{adj}}$ using $p$ as the query (Lines 1–4). The top-$M_{\mathrm{cross}}$ nearest objects are then stored as a separate cross-cell neighbor group of $p$ (Lines 5–6). As illustrated in Fig. 2, separating cross-cell from intra-cell neighbors allows query processing to selectively traverse cross-cell edges whose destination cells intersect the query filters.

**Implementation Optimization.** Algorithms 4–5 describe the standard construction procedure. To avoid redundant graph construction across layers, our implementation builds local graphs from scratch only at the finest layer and incrementally constructs coarser layers by reusing existing edges. Specifically, each parent cell inherits the intra-cell edges of its children and the cross-cell edges between sibling cells. We then introduce entry points for sibling cells and perform a synchronous refinement round to improve navigability within the parent cell. Reverse-edge repair is performed before constructing cross-cell edges at each layer. The construction-time proof appears in our technical report [47].

*Example 5.1.* Consider the 2D attribute space with $L = 3$ illustrated in the upper-left panel of Fig. 1. In Phase 1, each object is assigned to exactly one cell at each layer, and objects within the same non-empty cell are connected to form a local graph $\mathcal{G}_\ell^{\mathrm{intra}}$. In Phase 2, for each object $p$ in cell $c$, CubeGraph searches the local graphs of non-empty face-adjacent cells and connects $p$ to its nearest neighbors in each cell. These connections collectively form the cross-cell graph $\mathcal{G}_\ell^{\mathrm{cross}}$.

**Index Construction Time.** We analyze the time complexity of the two-phase index construction (Algorithms 4 and 5).

Theorem 5.2. *Assuming an average $O(\log N)$ cost per local graph insertion or search with fixed graph parameters, constructing the* CubeGraph *index takes $O(NLm \log N)$ time.*

**Remark.** Under this assumption, Phase 1 takes $O(NL \log N)$ time, while Phase 2 takes $O(NLm \log N)$ time due to searches across at most $2m$ face-adjacent cells per object. In practice, cross-cell searches use a smaller beam width (20 in our implementation) than local graph construction ($ef_c = 200$). Thus, their relative wall-clock costs depend on cell occupancy and parallelism.

## 5.2 Index Maintenance

In practice, objects may be inserted into or deleted from $\mathcal{D}$ after the CubeGraph index is constructed. Rebuilding the entire index for each update is costly, as it recomputes local graphs and cross-cell edges for all objects. Instead, we exploit the locality of updates: inserting or deleting an object affects only a limited number of cells. Accordingly, we develop incremental maintenance algorithms that update only the affected cells and their graph connections. We describe the insertion and deletion procedures below.

**Object Insertion.** When a new object $o = (\mathbf{x}, \mathbf{s})$ arrives, CubeGraph inserts it into every layer of the index. For each layer $\ell \in [0, L-1]$, CubeGraph determines the target cell from the attributes $\mathbf{s}$ using the assignment rule in Algorithm 4, creating a new cell in the sparse cell directory if necessary. The object is then inserted into the cell's local graph using the standard HNSW insertion procedure with parameter $M$, incrementally updating $\mathcal{G}_\ell^{\mathrm{intra}}$. Next, CubeGraph constructs the outgoing cross-cell neighbor groups of $o$ following Algorithm 5. Specifically, for each non-empty face-adjacent cell, it performs an A$k$NN search on the cell's local graph with $k = M_{\mathrm{cross}}$ and retains the results as a cross-cell neighbor group. Thus, each insertion adds at most $M$ intra-cell edges and $2mM_{\mathrm{cross}}$ outgoing cross-cell edges per layer, preserving the bounded-degree structure of CubeGraph.

**Object Deletion.** Physically removing an object $o = (\mathbf{x}, \mathbf{s})$ from CubeGraph requires repairing both intra-cell and cross-cell neighbor lists at every layer, incurring substantial maintenance overhead.

Table 2: The statistics of the datasets.

| Dataset | Size | Dim | Queries used | Attributes |
|---|---|---|---|---|
| SIFT1M | 1,000,000 | 128 | 1,000 | 2D/3D/4D Uniform |
| YFCC | 999,894 | 512 | 1,000 | 2D/3D Geolocation |
| WIKI | 982,119 | 384 | 1,000 | 2D/3D Geolocation |
| MSMARCO10M | 10,000,000 | 1024 | 1,000 | 2D/3D/4D Uniform |

**Table 3: Predetermined-to-on-the-fly QPS ratios at 99% recall@20; ratios above 1 favor predetermined search.**

| Dataset | 0.5% | 1% | 2% | 5% |
|---|---|---|---|---|
| SIFT1M 2D | 1.013× | 1.047× | 1.076× | 1.094× |
| MSMARCO10M 2D | 0.993× | 0.997× | 0.998× | 1.005× |

To avoid this cost, CubeGraph adopts a lazy deletion strategy that marks deleted objects as invalid without modifying the graph structure. Specifically, when an object $o$ is deleted, CubeGraph locates its corresponding cell at each layer $\ell \in [0, L-1]$ and marks it as invalid. During query processing, invalid objects are excluded from the top-$k$ results but remain available for graph traversal, preserving navigability and query correctness. Since adjacency lists remain unchanged, the deletion cost is proportional to the number of hierarchy layers $L$. However, accumulating invalid objects may gradually degrade search performance. To mitigate this, CubeGraph performs local compaction or rebuilds a cell when its invalid-object ratio exceeds a predefined threshold.

**Dynamic Update Complexity.** We analyze the time cost of incremental index maintenance under object insertions and deletions.

Theorem 5.3. *Assuming each local graph insertion or search takes $O(\log N)$ time, object insertion into* CubeGraph *takes $O(Lm \log N)$ time, while lazy deletion takes $O(L)$ time.*

# 6 EXPERIMENTS

Our experiments evaluate (1) the query efficiency of different methods (Exp-1), (2) the two search modes (Exp-2), (3) performance under complex geometric filters (Exp-3), (4) index construction costs (Exp-4), and (5) index maintenance (Exp-5). Our technical report [47] provides additional evaluations of selectivity, search modes, attribute distributions, filter aspect ratios, and index costs.

## 6.1 Experimental Setup

**Datasets.** We conduct experiments on four real-world datasets, whose sizes and vector dimensions are summarized in Table 2. SIFT1M [4] and MSMARCO10M use uniformly generated synthetic attributes in $[0, 1]^m$, where $m \in \{2, 3, 4\}$. The other two datasets use CLIP image embeddings and text embeddings, respectively, with real geolocation (2D) and timestamps (1D), supporting 2D and 3D attributes. The suffix "$m$D" indicates the attribute dimensionality, independent of the vector dimension.

**Baselines.** We compare CubeGraph with four existing methods for filtered A$k$NN search: (1) PostFiltering, a post-filtering method that searches the attribute-agnostic HNSW graph and filters results afterward; (2) ACORN-$\gamma$ [29] with $\gamma = 12$, a state-of-the-art filter-aware method; (3) iRangeGraph [43], a partition-based method designed for low-dimensional attributes; and (4) KHI [50], a state-of-the-art partition-based method for high-dimensional attributes. We compare all four baselines under hyperrectangular filters, while only ACORN and PostFiltering for complex geometric filters.

**Workloads and Metrics.** We generate random query vectors and geometric filters of different shapes. Hyperrectangular filters are generated with selectivities of 0.5/1/2/5/10%, defined as the fraction of objects satisfying the filter. We report 0.5–5% in the main text and 10% in our technical report [47]. Complex geometric filters use fixed, count-calibrated workloads.

For each workload, we generate 1,000 queries with exact ground truth ($k = 20$) and report average results. We measure effectiveness using recall@20 and efficiency using queries per second (QPS), averaged over three runs. We select the smallest beam width achieving at least 99% recall in every run. N/R indicates that no qualifying beam width is found, while INV denotes no filter-valid beam width. Beam widths range from 20 to 2048, extended to 4096 for ACORN and MSMARCO10M 3D/4D.

**Implementation and Execution.** All methods run on the same machine with 1.5 TiB RAM, with vectors and indexes stored in memory. All queries are executed using a single thread. CubeGraph is created using 128 threads. We first build the lowest-layer ($L-1$) graph with $M = 16$ and $ef_c = 200$. Higher layers reuse their child-layer graphs with refinement using a beam size of 64. Cross-cube edge searches use a beam size of 20 and retain two edges per occupied face-adjacent cube. We use predetermined search for hyperrectangular filters and on-the-fly search for other filters.

## 6.2 Experimental Results

**Exp-1: Comparison of Query Efficiency.** We evaluate the query efficiency of CubeGraph against KHI, iRangeGraph, ARCO, and PostFiltering under varying recall@20 levels, reporting queries per second (QPS) for hyperrectangle filters. Fig. 3 compares these methods on SIFT (2D attributes) and WIKI (3D attributes). Fig. 4 presents results on SIFT and YFCC (3D attributes), and SIFT and MSMARCO10M (4D attributes), excluding iRangeGraph, which fails on these datasets. We vary filter selectivity, defined as the fraction of objects satisfying the hyperrectangle filter, over 0.5%, 1%, 2%, and 5%, with additional results at 10% in our technical report [47].

The results show that CubeGraph is effective for highly selective multi-attribute filters. At 0.5% selectivity, CubeGraph outperforms KHI by 1.18–3.79× across all 3D/4D datasets, achieving 3.79/3.42× speedups on SIFT and 3.20/2.96× on MSMARCO10M with 3D/4D attributes, respectively. At 1%, CubeGraph remains 1.41–3.89× faster than KHI. Overall, at 0.5%, CubeGraph outperforms KHI in nine of ten settings, with YFCC (2D) being the only exception, where CubeGraph achieves 0.54× KHI's QPS. On SIFT (2D), CubeGraph achieves 1.91/1.86/1.46/0.94× KHI's QPS at 0.5/1/2/5% selectivity, respectively; on WIKI (3D), the corresponding ratios are 1.18/1.41/1.68/1.20×. At 10%, KHI leads on both datasets, with CubeGraph achieving 0.75/0.59× its QPS, respectively. KHI also generally leads on YFCC and MSMARCO10M (2D) at 1–10% selectivity. Notably, PostFiltering fails to reach 99% recall@20 at 0.5% or 1% selectivity for any hyperrectangle filter, highlighting CubeGraph's effectiveness in handling filter-mismatched objects.

**Exp-2: Effect of Search Modes.** We evaluate the two search modes, predetermined and on-the-fly, on SIFT1M and MSMARCO10M with

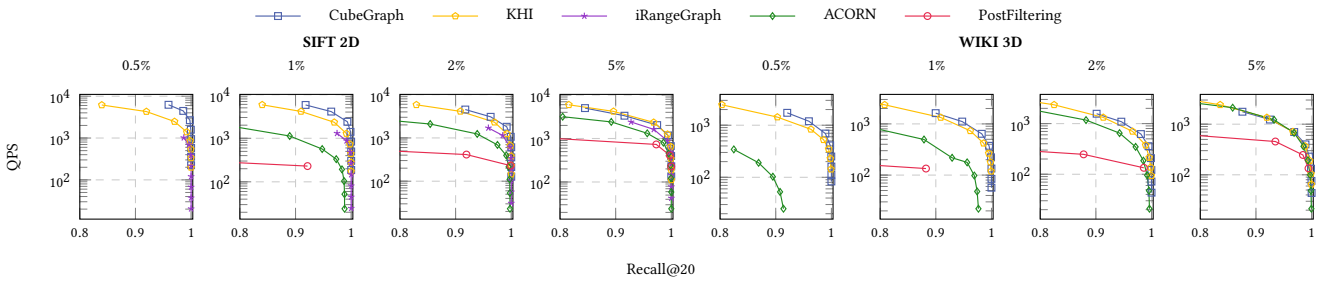



**Figure 3: Performance comparison of different methods on SIFT (2D) and WIKI (3D) with hyperrectangle filters.**

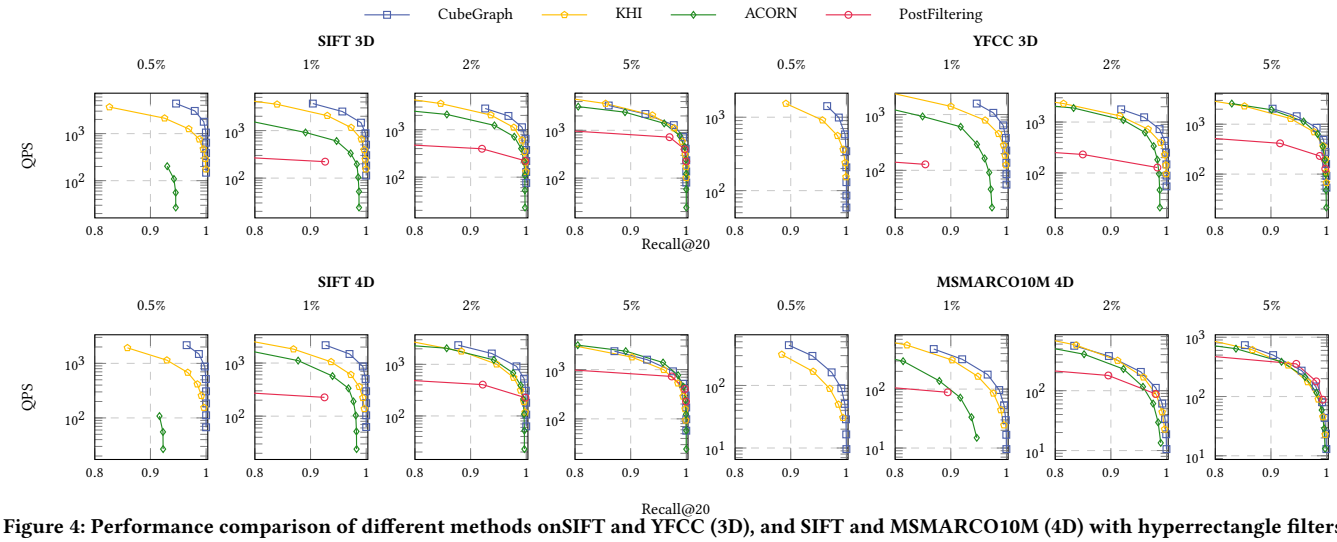



**Figure 4: Performance comparison of different methods onSIFT and YFCC (3D), and SIFT and MSMARCO10M (4D) with hyperrectangle filters.**

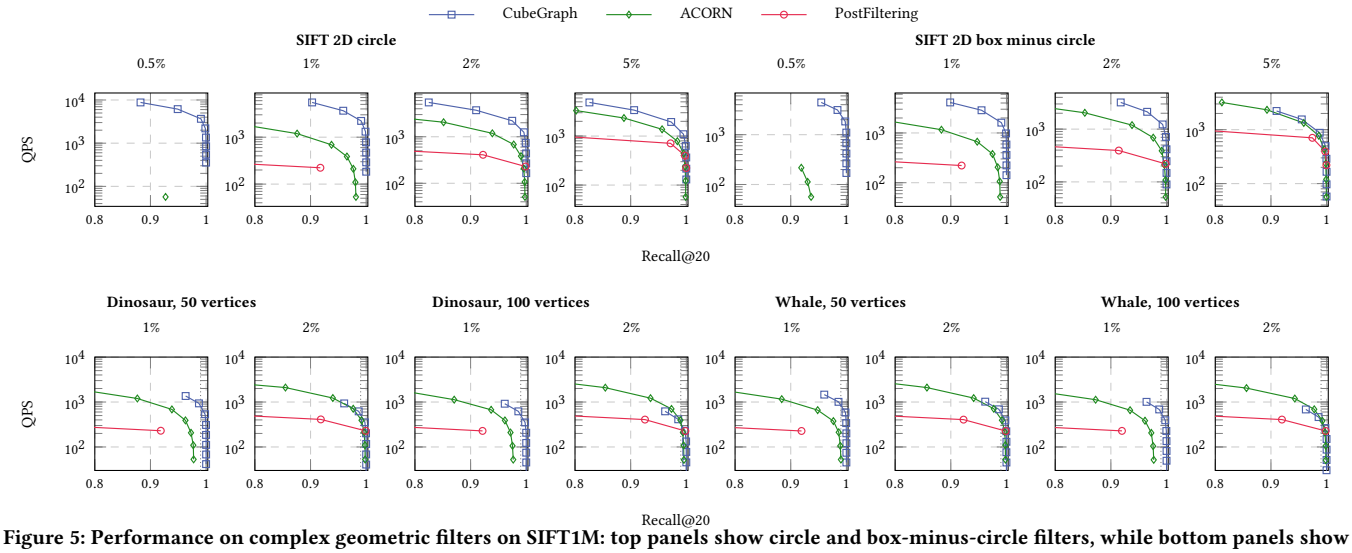



**Figure 5: Performance on complex geometric filters on SIFT1M: top panels show circle and box-minus-circle filters, while bottom panels show dinosaur- and whale-shaped filters.**

uniformly distributed 2D attributes at 1% and 5% selectivity. Table 3 reports their QPS ratios (predetermined over on-the-fly) at 99% recall@20, where ratios above 1 indicate that predetermined search is faster. Both modes use the same six-layer CubeGraph index and beam-width range (20–2048), ensuring a fair comparison.

The results show comparable performance under hyperrectangle filters. However, on-the-fly search generally performs better at higher selectivity ratios (e.g., 5%), as more filter-matching objects become available, making them easier to discover during traversal. This advantage increases with selectivity. Accordingly, we adopt

**Table 4: Comparison of index construction time and storage costs on MSMARCO10M with 2D attributes.**

| Method | Layers | Time (s) | CubeGraph | Size (GB) | CubeGraph |
|---|---|---|---|---|---|
| CubeGraph | 6 | 4,328.6 | 1.00× | 10.880 | 1.00× |
| KHI | 25 | 3,575.8 | 0.83× | 19.468 | 1.79× |
| iRangeGraph | 25 | 19,366.6 | 4.47× | 18.420 | 1.69× |

on-the-fly search for hyperrectangle filters, given its simple applicability and competitive efficiency, and predetermined search for other filter types. This choice achieves good overall performance. Additional experiments with different filter shapes in our technical report [47] confirm similar trends.

**Exp-3: Performance on Complex Geometric Filters.** We evaluate CubeGraph on complex geometric filters beyond hyperrectangles, comparing it with ACORN and PostFiltering, as other baselines cannot effectively support these filters.

First, we consider circle and box-minus-circle filters (objects inside a box but outside a circle), varying selectivity from 0.5% to 5%. Figure 5 (top) shows the recall@20–QPS trade-offs. CubeGraph consistently outperforms both baselines. At 0.5% selectivity, only CubeGraph achieves the target 99% recall, reaching 3,665 and 1,841 QPS for circle and box-minus-circle filters, respectively. This advantage persists at 1%. At 2%, CubeGraph is 3.25/3.16× faster than ACORN and 5.45/5.59× faster than PostFiltering for the two filter types, respectively. Similar trends hold for box-minus-circle filters across selectivities.

Second, we evaluate more complex polygonal filters (dinosaur and whale shapes, each with 50 and 100 vertices; specifications are provided in our technical report [47]) on SIFT1M at 1% and 2% selectivity. At 1%, CubeGraph achieves 99% recall on all four filters at 351–584 QPS. In contrast, ACORN reaches this recall only for the 50-vertex whale (53 QPS versus 584 for CubeGraph), while PostFiltering fails on all four. At 2%, neither CubeGraph nor ACORN consistently dominates, but CubeGraph remains competitive. Additional results on more complex filters and attribute distributions are provided in our technical report [47].

**Exp-4: Index Construction and Storage Costs.** We compare the index construction time and storage overhead of CubeGraph against KHI and iRangeGraph on MSMARCO10M with 2D attributes. CubeGraph uses six layers, while both baselines use their default 25 layers. Table 4 reports the results.

For construction time, KHI is the fastest, taking 0.83× that of CubeGraph, while iRangeGraph requires 4.47× as much time. Thus, CubeGraph achieves competitive construction efficiency. For storage, CubeGraph occupies only 10.880 GB, reducing space by 44.1% and 40.9% compared with KHI and iRangeGraph, respectively. Additional results across all datasets and higher attribute dimensions are available in our technical report [47].

**Exp-5: Dynamic Index Maintenance.** We evaluate the proposed incremental maintenance method by inserting batches of 10/30/50% new objects into an initial index of 100K vectors drawn from a 200K pool on SIFT1M and YFCC. Table 5 reports the results.

Incremental maintenance is 1.89–6.18× faster than rebuilding the index on the resulting dataset, although the advantage decreases with larger batches. At 10% insertion, it achieves 5.96× and 6.18×

**Table 5: Incremental update performance on SIFT1M and YFCC, where Ratio = rebuild time / incremental update time.**

| | SIFT1M | | | YFCC | | |
|---|---|---|---|---|---|---|
| Batch | Inc. | Rebuild | Ratio | Inc. | Rebuild | Ratio |
| 10% | 0.78 | 4.66 | 5.96× | 1.97 | 12.14 | 6.18× |
| 30% | 2.04 | 5.39 | 2.64× | 5.64 | 15.77 | 2.80× |
| 50% | 3.74 | 7.07 | 1.89× | 9.75 | 20.19 | 2.07× |

speedups on SIFT1M and YFCC, respectively. These results demonstrate the efficiency of CubeGraph in supporting dynamic updates.

## 7 RELATED WORK

§ 2.2 reviewed the foundations of A$k$NN search and index merging. Here, we focus on filtered A$k$NN search with numeric and categorical filters, which is most closely related to our problem of A$k$NN search with geometric filters.

**Numeric-Filtered A$k$NN Search.** This line of research considers A$k$NN search with numeric range filters [16, 23, 31, 39, 44, 45, 51, 54]. Most existing methods focus on one-dimensional range filters and organize or combine indexes over overlapping subsets of objects. For example, SeRF [54] compresses the $O(N^2)$ indexes associated with all possible range-filtered subsets into a compact structure. Segment-tree-based methods [8, 40, 43] organize numeric intervals hierarchically, with each node indexing objects within its corresponding range. Hi-PNG [44] also employs hierarchical partitioning but targets interval-filtered ANNS, where both data and query vectors are associated with numeric intervals. While effective for range and interval filters, these methods do not naturally support arbitrary multidimensional geometric filters. In contrast, CubeGraph adaptively selects a cell resolution and exploits precomputed face-adjacent connections for graph navigation, while enforcing exact geometric filters during candidate verification.

**Categorical-Filtered A$k$NN Search.** This line of research considers A$k$NN search with categorical filters, such as labels. Existing methods typically construct or combine graph indexes based on label categories [13, 22, 25, 35, 42, 48, 52]. For example, UNG [7] constructs label-specific graphs and connects them through cross-label edges. UniFilter [42] introduces an automaton-guided navigation graph over a base index to support categorical filtering. Unlike these methods, which target discrete label filters, CubeGraph supports continuous geometric filters by covering query filter with cells at an appropriate resolution and exploiting cross-cell edges for graph navigation.

## 8 CONCLUSION

This paper presents CubeGraph, a hierarchical graph index for A$k$NN search with geometric filters. CubeGraph combines a bounded-depth, multi-resolution cell hierarchy with cross-cell edges to balance selectivity and navigability. For query processing, we develop a principled layer-selection strategy and two search methods: precomputed and on-the-fly search. We further design efficient index construction and dynamic maintenance algorithms. Extensive experiments on real-world datasets demonstrate that CubeGraph efficiently and effectively supports diverse geometric filters. Future work includes distributed index construction and improved query processing techniques.

# REFERENCES


[1] 2026. Zekai Wu, Jiabao Jin, Peng Cheng, Xiaoyao Zhong, Lei Chen, Yongxin Tong, Zhitao Shen, Jingkuan Song, Heng Tao Shen, Xuemin Lin. *arXiv preprint arXiv:2603.21710* (2026). https://arxiv.org/abs/2603.21710

[2] Md Mahbub Alam, Luis Torgo, and Albert Bifet. 2022. A survey on spatio-temporal data analytics systems. *Comput. Surveys* 54, 10s (2022), 1–38.

[3] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2015. Cache locality is not enough: High-Performance Nearest Neighbor Search with Product Quantization Fast Scan. *Proceedings of the VLDB Endowment* 9, 4 (2015).

[4] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems* 87 (2020), 101374.

[5] Artem Babenko and Victor Lempitsky. 2014. Additive quantization for extreme vector compression. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 931–938.

[6] Norbert Beckmann, Hans-Peter Kriegel, Ralf Schneider, and Bernhard Seeger. 1990. The R*-tree: An efficient and robust access method for points and rectangles. In *Proceedings of the 1990 ACM SIGMOD international conference on Management of data*. 322–331.

[7] Yuzheng Cai, Jiayang Shi, Yizhuo Chen, and Weiguo Zheng. 2024. Navigating Labels and Vectors: A Unified Approach to Filtered Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 1–27.

[8] Joshua Engels, Benjamin Landrum, Shangdi Yu, Laxman Dhulipala, and Julian Shun. 2024. Approximate Nearest Neighbor Search with Window Filters. *ICML 2024* (2024).

[9] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proc. VLDB Endow.* 12, 5 (2019), 461–474.

[10] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.

[11] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2014. Optimized Product Quantization. *IEEE Trans. Pattern Anal. Mach. Intell.* 36, 4 (2014), 744–755.

[12] Aristides Gionis, Piotr Indyk, Rajeev Motwani, et al. 1999. Similarity search in high dimensions via hashing. In *Vldb*, Vol. 99. 518–529.

[13] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, et al. 2023. Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*. 3406–3416.

[14] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the thirtieth annual ACM symposium on Theory of computing*. 604–613.

[15] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in Neural Information Processing Systems* 32 (2019).

[16] Mengxu Jiang, Zhi Yang, Fangyuan Zhang, Guanhao Hou, Jieming Shi, Wenchao Zhou, Feifei Li, and Sibo Wang. 2025. DIGRA: A Dynamic Graph Indexing for Approximate Nearest Neighbor Search with Range Filter. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.

[17] Chenzhe Jin, Yunan Zhang, Jiayi Liu, and Jianguo Wang. 2026. Efficient Vector Index Merging in Vector Databases. 4, 1 (2026). https://doi.org/10.1145/3786645

[18] Liuchang Jing, Mingyu Yang, Lei Li, Jianbin Qin, and Wei Wang. 2026. Multiple Index Merge for Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2602.17099* (2026).

[19] Leonardo Kuffo, Elena Krippner, and Peter Boncz. 2025. PDX: A data layout for vector similarity search. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.

[20] Binhong Li, Xiao Yan, and Shangqi Lu. 2025. Fast-Convergent Proximity Graphs for Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2510.05975* (2025).

[21] Mocheng Li, Xiao Yan, Baotong Lu, Yue Zhang, James Cheng, and Chenhao Ma. 2025. Attribute Filtering in Approximate Nearest Neighbor Search: An In-depth Experimental Study. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–26.

[22] Zhaoheng Li, Silu Huang, Wei Ding, Yongjoo Park, and Jianjun Chen. 2025. SIEVE: Effective Filtered Vector Search with Collection of Indexes. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4723–4736.

[23] Anqi Liang, Pengcheng Zhang, Bin Yao, Zhongpu Chen, Yitong Song, and Guangxu Cheng. 2024. UNIFY: Unified Index for Range Filtered Approximate Nearest Neighbors Search. *Proceedings of the VLDB Endowment* 18, 4 (Dec. 2024), 1118–1130. https://doi.org/10.14778/3717755.3717770

[24] Kejing Lu, Mineichi Kudo, Chuan Xiao, and Yoshiharu Ishikawa. 2021. HVS: Hierarchical Graph Structure Based on Voronoi Diagrams for Solving Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 15, 2 (2021), 246–258.

[25] Jiarui Luo, Miao Qiao, Chaoji Zuo, and Dong Deng. 2025. Tag-Filtered Approximate Nearest Neighbor Search. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 3642–3654.

[26] Yu A Malkov and Dmitry A Yashunin. [n.d.]. hnswlib. https://github.com/nmslib/hnswlib.

[27] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836.

[28] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Vector Database Management Techniques and Systems. In *Companion of the 2024 International Conference on Management of Data*. 597–604.

[29] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. Acorn: Performant and predicate-agnostic search over vector embeddings and structured data. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.

[30] Yun Peng, Byron Choi, Tsz Nam Chan, Jianye Yang, and Jianliang Xu. 2023. Efficient Approximate Nearest Neighbor Search in Multi-dimensional Databases. *Proc. ACM Manag. Data* 1, 1 (2023), 54:1–54:27. https://doi.org/10.1145/3588908

[31] Zhencan Peng, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2025. Dynamic Range-Filtering Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* 18, 10 (June 2025), 3256–3268. https://doi.org/10.14778/3748191.3748193

[32] Parikshit Ram and Kaushik Sinha. 2019. Revisiting kd-tree for nearest neighbor search. In *Proceedings of the 25th acm sigkdd international conference on knowledge discovery & data mining*. 1378–1388.

[33] Yitong Song, Bin Yao, Zhida Chen, Xin Yang, Jiong Xie, Feifei Li, and Mengshi Chen. 2025. Efficient top-k spatial-range-constrained approximate nearest neighbor search on geo-tagged high-dimensional vectors. *The VLDB Journal* 34, 1 (2025), 14.

[34] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. 2021. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data*. 2614–2627.

[35] Mengzhao Wang, Lingwei Lv, Xiaoliang Xu, Yuxiang Wang, Qiang Yue, and Jiongkang Ni. 2022. Navigable proximity graph-driven native hybrid queries with structured and unstructured constraints. *arXiv preprint arXiv:2203.13601* (2022).

[36] Mengzhao Wang, Haotian Wu, Xiangyu Ke, Yunjun Gao, Yifan Zhu, and Wenchao Zhou. 2025. Accelerating Graph Indexing for ANNS on Modern CPUs. *arXiv preprint arXiv:2502.18113* (2025).

[37] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *Proc. ACM Manag. Data* 2, 1 (2024), V2mod014:1–V2mod014:27. https://doi.org/10.1145/3639269

[38] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proceedings of the VLDB Endowment* 14, 11 (2021), 1964–1978.

[39] Yuxiang Wang, Ziyuan He, Yongxin Tong, Zimu Zhou, and Yiman Zhong. 2025. Timestamp Approximate Nearest Neighbor Search over High-Dimensional Vector Data. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 3043–3055.

[40] Ziqi Wang, Jingzhe Zhang, and Wei Hu. 2025. WoW: A Window-to-Window Incremental Index for Range-Filtering Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–27.

[41] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. AnalyticDB-V: a hybrid analytical engine towards query fusion for structured and unstructured data. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3152–3165.

[42] Jiadong Xie, Jeffrey Xu Yu, Siyi Teng, and Yingfan Liu. 2025. Beyond Vector Search: Querying With and Without Predicates. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–26.

[43] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S Jensen. 2024. iRangeGraph: Improvising Range-dedicated Graphs for Range-filtering Nearest Neighbor Search. *arXiv preprint arXiv:2409.02571* (2024).

[44] Ming Yang, Yuzheng Cai, and Weiguo Zheng. 2025. Hi-PNG: Efficient Interval-Filtering ANNS via Hierarchical Interval Partition Navigating Graph. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 3518–3529.

[45] Mingyu Yang, Wentao Li, Zhitao Shen, Chuan Xiao, and Wei Wang. 2025. ESG: Elastic Graphs for Range-Filtering Approximate k-Nearest Neighbor Search. *arXiv preprint arXiv:2504.04018* (2025).

[46] Mingyu Yang, Wentao Li, and Wei Wang. 2024. Fast High-dimensional Approximate Nearest Neighbor Search with Efficient Index Time and Space. *arXiv preprint arXiv:2411.06158* (2024).

[47] Mingyu Yang, Wentao Li, Wenxuan Xia, and Wei Wang. 2026. *CubeGraph: A Multi-Resolution Graph Index for Approximate k-Nearest Neighbor Search with Geometric Filters*. Technical Report. HKUST (GZ). https://github.com/mingyu-hkustgz/CubeGraph/blob/main/technical_report.pdf

[48] Mingyu Yang, Wenxuan Xia, Wentao Li, Raymond Chi-Wing Wong, and Wei Wang. 2025. Elastic Index Select for Label-Hybrid Search in Vector Database. *arXiv preprint arXiv:2505.03212* (2025).

[49] Ziqi Yin, Jianyang Gao, Pasquale Balsebre, Gao Cong, and Cheng Long. 2025. DEG: Efficient Hybrid Vector Search Using the Dynamic Edge Navigation Graph. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–28.

[50] Yuanhang Yu, Dawei Cheng, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2026. Efficient Approximate Nearest Neighbor Search under Multi-Attribute Range Filter. *arXiv preprint arXiv:2602.15488* (2026).

[51] Fangyuan Zhang, Mengxu Jiang, Guanhao Hou, Jieming Shi, Hua Fan, Wenchao Zhou, Feifei Li, and Sibo Wang. 2025. Efficient Dynamic Indexing for Range Filtered Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.

[52] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, et al. 2023. {VBASE}: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 377–395.

[53] Xiaoyao Zhong, Haotian Li, Jiabao Jin, Mingyu Yang, Deming Chu, Xiangyu Wang, Zhitao Shen, Wei Jia, George Gu, Yi Xie, et al. 2025. VSAG: An Optimized Search Framework for Graph-based Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2503.17911* (2025).

[54] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2024. SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–26.

# A PROOFS

**Proof of Theorem 4.4.** Each object has one record per index layer. With fixed intra-cell and cross-cell degree parameters $M$ and $M_{\text{cross}}$ (We treat them as constants), each record stores $O(1)$ intra-cell edges, $O(m)$ cross-cell edges to at most $2m$ face-adjacent cells, and $O(m)$ attributes, following our physical layout. Thus, all records require $O(LNm)$ space. Each layer contains at most $N$ occupied cells, whose coordinates and adjacency metadata require an additional $O(LNm)$ space. Therefore, the total space complexity is $O(LNm)$ for $m \geq 1$.

**Proof of Proposition 4.6.** In CubeGraph, every cell intersecting $\phi$ also intersects its bounding box $B(\phi)$. Along dimension $i$, a bounding-box side of length $a_i$ intersects at most $\lceil a_i/w_{\ell^*} \rceil + 1$ cells. Since $a_i \leq r < 2w_{\ell^*}$, this is at most three cells per dimension, yielding at most $3^m$ intersected cells.

**Proof of Lemma 4.8.** Let $\phi$ be a ball of radius $r/2$, and let $U$ be its covering region at the selected layer $\ell^*$. The volume elastic factor measures the fraction of $U$ occupied by $\phi$:

$$\bar{e} = \frac{\text{vol}(\phi)}{\text{vol}(U)}.$$

By Proposition 4.6, the selected cell width satisfies $w_{\ell^*} \leq r$, and at most $3^m$ cells are needed to cover $\phi$. Thus, the total covering volume is bounded by

$$\text{vol}(U) \leq 3^m w_{\ell^*}^m \leq 3^m r^m.$$

Since $\phi$ lies entirely within $[0,1]^m$, its volume is

$$\text{vol}(\phi) = \frac{\pi^{m/2}}{\Gamma(m/2+1)} \left(\frac{r}{2}\right)^m.$$

Combining the two volume bounds, we obtain

$$\bar{e} \geq \frac{\pi^{m/2}(r/2)^m}{3^m r^m \Gamma(m/2+1)} = \frac{\pi^{m/2}}{6^m \Gamma(m/2+1)}.$$

Therefore, the volume elastic factor has a lower bound that depends only on the dimension $m$, regardless of the ball radius $r/2$.

**Proof of Theorem 4.9.** Binary search identifies the query layer in $O(\log L)$ time, followed by cell discovery costing $T_{\text{cube}}$ and expected startup cost $C$. Conditioned on any traversal history, the probability that the next $t$ candidates are all invalid is at most $(1-c)^t$. Thus, the expected waiting time per valid candidate is at most $1/c$, yielding $k/c$ for $k$ valid candidates. Under the assumed bounded per-candidate cost and continued traversal, the total expected time is $O(\log L + T_{\text{cube}} + C + k/c)$. Predetermined discovery takes $O(3^m)$ time with constant-time intersection tests, while on-the-fly checks are charged to traversal. Variable queue and predicate costs must be accounted for separately. This bound concerns valid-candidate retrieval, not nearest-neighbor recall.

**Proof of Theorem 5.2.** Assuming an average local insertion/search cost of $O(\log N)$, local index construction takes $O(NL \log N)$ time. Each object searches at most $2m$ adjacent cells per layer, adding $O(NLm \log N)$ time. Thus, the total construction time is $O(NLm \log N)$. This is a conditional average-case bound, not a worst-case HNSW guarantee.

**Proof of Theorem 5.3.** For insertion, each layer requires $O(m)$ time to locate the cell, $O(\log N)$ for local insertion, and $O(m \log N)$ to search at most $2m$ adjacent cells. Thus, insertion across $L$ layers takes $O(Lm \log N)$ time under the assumed average search cost. For deletion, the maintained object-to-cell mapping allows direct marking of one record per layer, taking $O(L)$ time. Subsequent compaction is excluded.

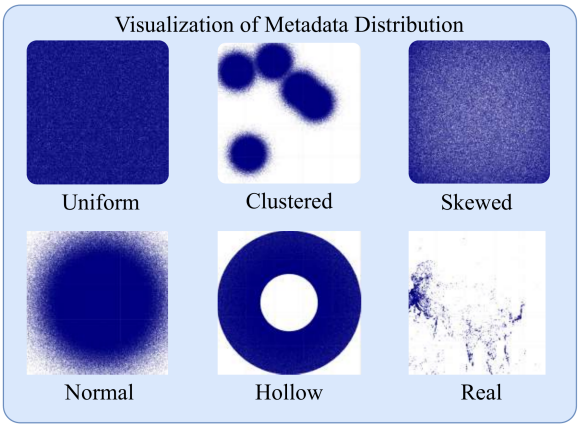



**Figure 6: Illustration of attribute distributions: the first five are synthetic, and the last uses real attributes.**

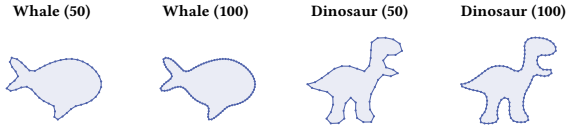



**Figure 7: Illustration of complex geometric filters using whale and dinosaur polygon templates with 50 and 100 vertices, respectively.**

# B ADDITIONAL EXPERIMENTAL RESULTS

We first describe the attribute distributions and filter shapes used in our experiments, followed by additional experimental results.

**Attribute Distributions and Filter Shapes.** Attribute distributions affect query performance in geometric-filtered A$k$NN search. Figure 6 illustrates the synthetic and real attribute distributions used in our experiments. The main comparisons use uniform synthetic and real attributes, while Section AExp-3 further evaluates four nonuniform synthetic distributions.

We also illustrate the complex filter shapes used in our experiments. Figure 7 shows the exact whale and dinosaur polygon templates used in the workload, displayed at a common scale with shaded interiors. These templates are scaled and translated to generate individual queries.

**AExp-1: Impact of Selectivity.** While the main text considers filter selectivity up to 5%, we further evaluate hyperrectangle filters at 10% selectivity on SIFT1M, YFCC, WIKI, and MSMARCO10M with 2D and 3D attributes, following the setup in Section 6.1. Figure 8 shows that all methods (CubeGraph, KHI, iRangeGraph, ACORN, and PostFiltering ) achieve 99% recall, while throughput varies across methods and datasets. Although CubeGraph and KHI do

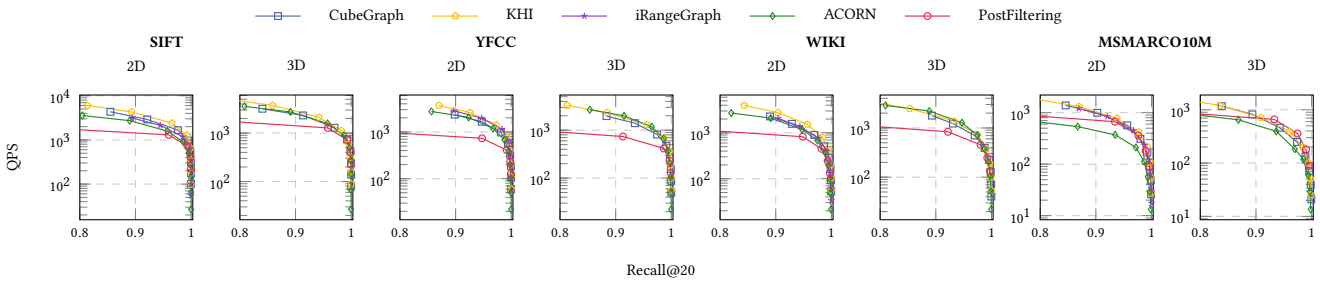



**Figure 8: Performance comparison of methods on SIFT, YFCC, WIKI, and MSMARCO10M using hyperrectangle filters with 10% selectivity.**

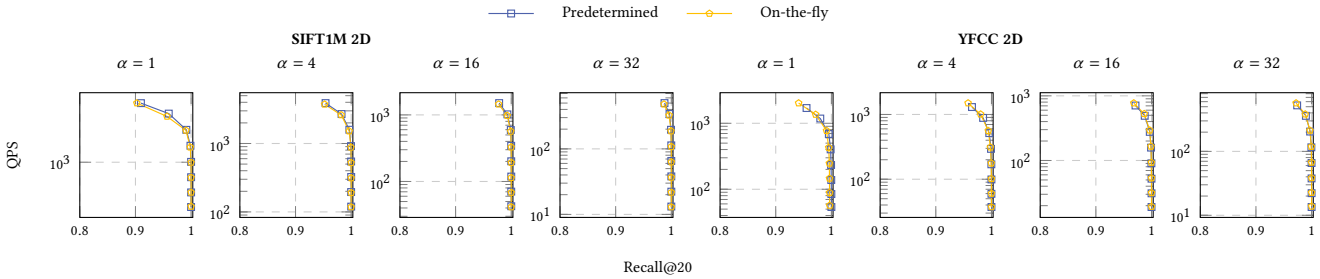



**Figure 9: Effect of the long-to-short side ratio on search modes for SIFT1M and YFCC at 1% selectivity.**

**Table 6: Comparison of index construction time (seconds) and space (decimal GB).**

| Dataset | Dim. | CubeGraph | | KHI | | iRangeGraph | | ACORN | | POST | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | Time | Space | Time | Space | Time | Space | Time | Space | Time | Space |
| SIFT1M | 2D | 65.2 | 1.088 | 63.3 | 1.399 | 433.0 | 1.309 | 88.4 | 1.022 | 521.1 | 0.140 |
| | 3D | 52.0 | 0.812 | 63.6 | 1.399 | — | — | 88.4 | 1.022 | 534.7 | 0.140 |
| YFCC | 2D | 167.6 | 1.088 | 98.6 | 1.338 | 836.2 | 1.282 | 176.5 | 2.557 | 953.9 | 0.140 |
| | 3D | 101.2 | 0.812 | 97.4 | 1.455 | — | — | 176.5 | 2.557 | 937.0 | 0.140 |
| WIKI | 2D | 159.3 | 1.069 | 103.5 | 1.657 | 809.3 | 1.574 | 215.5 | 2.009 | 752.7 | 0.137 |
| | 3D | 98.6 | 0.798 | 133.0 | 1.655 | — | — | 215.5 | 2.009 | 724.1 | 0.137 |
| MSMARCO10M | 2D | 4,328.6 | 10.880 | 3,575.8 | 19.468 | 19,366.6 | 18.420 | 4,258.6 | 46.056 | 10,609.4 | 1.400 |
| | 3D | 3,328.6 | 10.122 | 3,574.3 | 19.468 | — | — | 4,258.6 | 46.056 | 10,500.2 | 1.400 |

Dashes indicate unmeasured 3D iRangeGraph configurations. ACORN reuses a single vector index per dataset across dimensions. Timing scopes, thread counts, and serialization formats vary across methods; reported sizes exclude external input files.

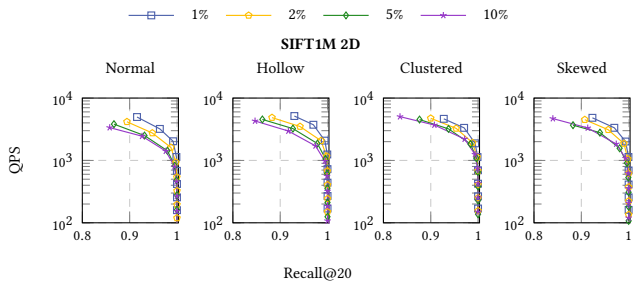



**Figure 10: Impact of attribute distribution on performance.**

not consistently achieve the highest throughput, both remain competitive at 10% selectivity. These results confirm that CubeGraph remains effective for hyperrectangle filters with higher selectivity.

**AExp-2: Impact of Aspect Ratio on Search Modes.** We evaluate predetermined and on-the-fly search on SIFT1M and YFCC with 2D attributes, fixing selectivity at 1% and varying the long-to-short side ratio $\alpha = 1, 4, 16, 32$ (Figure 9). The two modes use identical indexes and frozen queries at each aspect ratio. Each curve reports median recall@20 and QPS over three runs of 1,000 queries with $k = 20$ and beam widths 20–2048.

Both modes reach 99% recall in every run. We compare QPS at the smallest beam width reaching this recall floor in all three runs. Predetermined search is 1.9–4.8% faster on SIFT1M, whereas on-the-fly search is 4.0–12.3% faster on YFCC across the four aspect ratios. For predetermined search, QPS drops from 2,895 to 341 on SIFT1M and from 688 to 204 on YFCC as $\alpha$ increases from 1 to 32. Thus, elongated filters reduce throughput while preserving high recall, and the faster search mode depends on the dataset.

**AExp-3: Impact of Attribute Distribution.** In previous experiments, we considered uniformly distributed and real-world attributes. Here, we further evaluate CubeGraph on SIFT1M with four additional 2D attribute distributions: Normal, Hollow, Clustered, and Skewed (Figure 6). We vary selectivity from 1% to 10% (1%, 2%, 5%, and 10%) and use on-the-fly search, with results shown in Figure 10. CubeGraph achieves 99% recall across all distributions and selectivities. At 1% selectivity, QPS ranges from 1,880 (Clustered) to 2,079 (Hollow), a 10.6% difference between the highest and

lowest values. These results demonstrate that CubeGraph maintains high recall across diverse attribute distributions, with modest throughput variation.

**AExp-4: Index Construction Time and Space (Full Results).** The main text reports index construction time and space on MS-MARCO10M with 2D attributes. Here, we extend the evaluation to all four datasets, covering both 2D and 3D attributes, and compare CubeGraph with KHI, iRangeGraph, ACORN, and PostFiltering. Table 6 presents the results. We use 128 construction threads for CubeGraph, 32 for KHI and iRangeGraph, 72 for ACORN, and one for PostFiltering. Construction times exclude disk writes except for PostFiltering, while CubeGraph additionally includes input reading. Index sizes reflect native serialized files, which contain different embedded data across methods, rather than total serving memory.

CubeGraph requires 18.7–51.8% less serialized space than KHI across all settings and 15.1–40.9% less than iRangeGraph across all four measured 2D configurations. The reported construction times reflect each method's native settings, and the results are consistent with those in the main text.